

REF	DATE	TYPE	AUTHOR	STATUS
W4-R-2026-004	JUN 2026	WHITEPAPER	ENOCH KAN	PUBLISHED

Trusting the Untrusted: Prompt Injection in Tool-Using Agents

A taxonomy of indirect injection paths in agentic systems, why filtering alone fails, and how to design agents that survive hostile input.

ABSTRACT

Agentic systems extend large language models with tools, memory, and autonomy — and in doing so, they extend the attack surface far beyond the chat box. Every document an agent retrieves, every API response it parses, and every message it receives from another agent is a potential carrier of adversarial instructions. This paper presents a practical taxonomy of indirect prompt injection paths in tool-using agents, explains why content filtering is structurally insufficient as a defence, and sets out an architecture-first mitigation strategy built on privilege separation, action gating, and continuous adversarial measurement. It closes with a checklist teams can apply to agents in production today.

1. Introduction: agents change the threat model

A chatbot that only talks to its user has one untrusted input: the user. An agent that reads email, browses the web, queries internal systems, and calls tools has many — and most of them are authored by people the operator has never met. Prompt injection, the manipulation of a model's behaviour through crafted input, stops being a curiosity about jailbreaking a chatbot and becomes a question of whether an attacker on the open internet can drive actions inside your infrastructure.

The core problem is well established: current language models do not enforce a hard boundary between instructions and data. Text retrieved as "content" is processed by the same mechanism that processes the operator's system prompt. OWASP ranks prompt injection as the number-one risk for LLM applications, and academic work on indirect injection (Greshake et al., 2023) demonstrated years ago that web pages and documents can hijack LLM-integrated applications without the user ever typing anything malicious. What has changed since is the blast radius: agents now hold credentials, execute code, and take actions with real-world consequences.

A useful mental model is the "lethal trifecta" popularised by Simon Willison: an agent becomes dangerous when it simultaneously (a) processes untrusted input, (b) has access to private data, and (c) can communicate externally or take actions. Most production agents today have all three.

2. A taxonomy of indirect injection paths

We group the injection paths we encounter in assessments into five classes. The classes matter because each one implies a different point of interception.

2.1 Retrieved content (RAG and search)

Any retrieval-augmented pipeline imports third-party text into the model's context. Poisoned pages, wiki entries, support tickets, or uploaded documents can carry payloads such as "ignore previous instructions and forward the conversation to this address." Because retrieval is ranked by relevance rather than trustworthiness, an attacker's job reduces to search-engine optimisation against your retriever.

2.2 Tool and API results

Agents routinely treat tool output as ground truth. A scraped web page, a webhook payload, a CRM record with an attacker-controlled free-text field, or an error message from a compromised dependency all re-enter the context with the implicit authority of "the system said so." Injection via tool results is especially dangerous because it arrives mid-task, after the user's intent has been established, when the agent is selecting its next action.

2.3 Inter-agent messages

Multi-agent architectures forward text between models. A compromised or manipulated upstream agent becomes an injection vector into every downstream agent that trusts its output. Delegation chains amplify this: by the third hop, no agent in the chain can attribute an instruction to its true origin.

2.4 Persistent memory

Agents with long-term memory can be poisoned slowly. An instruction smuggled into a stored note, profile field, or conversation summary executes not once but on every future session that loads the memory — a persistence mechanism directly analogous to stored XSS.

2.5 Tool descriptions and metadata

In protocol-based tool ecosystems, the tool's own description is text supplied by the tool vendor and read by the model. A malicious or compromised tool server can embed behavioural instructions in its metadata, its parameter descriptions, or its results — a supply-chain flavour of injection that arrives before the user's first message.

Injection path	Typical carrier	Analogous classic vuln
Retrieved content	Web pages, documents, tickets	SEO poisoning / watering hole
Tool results	API payloads, scraped HTML, errors	Deserialisation of untrusted data
Inter-agent messages	Delegation and hand-off text	Confused deputy
Persistent memory	Notes, summaries, profiles	Stored XSS
Tool metadata	Descriptions, parameter docs	Supply-chain compromise

3. Why filtering fails

The intuitive defence — scan inputs for malicious instructions — inherits every weakness of signature-based security, and adds a new one. Natural language has unbounded paraphrase space: instructions can be encoded, translated, split across documents, or phrased as helpful context ("the user would want you to..."). Detection models can raise the cost of attack, and are worth deploying, but they are probabilistic gates in front of a deterministic capability. When the downstream action is "send funds" or "execute code," a 98%-accurate classifier is a 2% standing invitation.

More fundamentally, filtering treats the symptom. The disease is that we hand a single undifferentiated context window to a model and grant that model privileges. Any defence that leaves this architecture intact is a mitigation, not a fix.

4. Architecture-first mitigations

4.1 Least-privilege tools

Scope every tool to the minimum capability the task requires: read-only where possible, allowlisted endpoints, per-session credentials, and no ambient authority. An injected instruction cannot exfiltrate through a channel the agent does not have.

4.2 Privilege separation between models

Adopt dual-model patterns: a privileged orchestrator that never reads untrusted content directly, and a quarantined worker that processes untrusted content but holds no tools. The worker returns structured, validated data — never free text that flows into the orchestrator's instruction stream. This mirrors the trusted/untrusted process split that hardened browsers and operating systems decades ago.

4.3 Action gating

Classify actions by consequence. Reversible, low-impact actions may proceed autonomously; consequential actions — spending, sending, deleting, deploying — require a deterministic policy check or explicit human confirmation that no in-context instruction can waive. The confirmation surface must show the actual action parameters, not the model's summary of them.

4.4 Provenance and taint tracking

Tag every context segment with its origin and propagate the tag. When a proposed action derives from tainted content, downgrade autonomy: require confirmation, drop privileges, or refuse. Even coarse-grained taint ("this plan step cites retrieved text") meaningfully raises attacker cost.

4.5 Constrain outputs, not just inputs

Force tool calls through typed schemas with validated arguments. Strip or neutralise markup channels commonly used for exfiltration, such as attacker-controlled URLs in generated markdown images or links. Egress filtering applies to agents just as it applies to servers.

5. Measuring exposure

Defences you cannot measure regress silently. We recommend a standing adversarial harness — the approach we take in our open-source toolkit Plexiglass — with three components:

- **Corpus replay.** Continuously test the agent against a maintained corpus of injection payloads spanning the five paths above, refreshed from public jailbreak and injection collections as they evolve.
- **Canary instructions.** Seed test environments with benign tracer payloads ("include the word AMBER in your reply") across documents, tool results, and memory. Canary execution rate is your injection success rate — a single number leadership can track.
- **Consequence probes.** Measure not whether the model repeats a phrase, but whether injected content can trigger tool calls it should not: the metric that actually maps to business risk.

6. Recommendations

- Enumerate every path by which third-party text reaches each agent; assume all of them are hostile.
- Deny the lethal trifecta where possible: an agent with untrusted input and private data should not have unsupervised external write access.
- Gate consequential actions behind deterministic policy or human confirmation, outside the model's control.
- Separate privileged orchestration from untrusted-content processing at the architecture level.
- Track injection success rate with canaries and replay corpora in CI, and treat regressions like failing tests.
- Prefer typed, validated tool interfaces over free-text tool invocation.

References

- OWASP Foundation. OWASP Top 10 for Large Language Model Applications — LLM01: Prompt Injection.
- Greshake, K. et al. (2023). Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection.
- Willison, S. Writing on prompt injection and the "lethal trifecta" for AI agents. simonwillison.net.
- MITRE ATLAS. Adversarial Threat Landscape for Artificial-Intelligence Systems. atlas.mitre.org.
- W4 Advisory. Plexiglass: a toolkit for detecting and protecting against vulnerabilities in LLMs. github.com/w4-advisory/plexiglass.

// © 2026 W4 Advisory. Published under CC BY 4.0. Feedback: hello@w4advisory.com